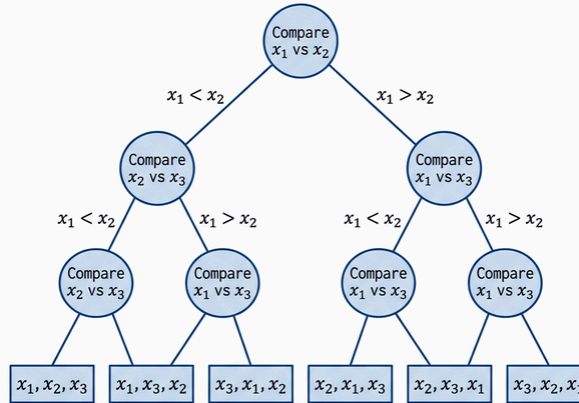


Doğrusal ve Hibrit Sıralama Algoritmaları

Hafta 4

Karşılaştırma Tabanlı Sıralama Sınırları

- Karşılaştırma tabanlı algoritmalar $\Omega(n \log n)$ alt sınırına sahiptir.
- Karar Ağacı (Decision Tree) modeli bu sınırı ispatlar.
- n elemanlı dizinin $n!$ kadar farklı dizilim ihtimali vardır.
- Bilgi teorisine göre $n \log n$ bir fiziksel limittir.



Decision Tree for Sorting $n = 3$ Elements

- Number of permutations (leaves) = $n! = 3! = 6$
- Minimum height of a binary tree with L leaves is $\lceil \log_2 L \rceil$
- Height $h \geq \lceil \log_2(n!) \rceil \approx n \log_2 n$
- $\therefore$ Sorting lower bound is $\Omega(n \log n)$

Non-Comparison Based Sorting Giriş

- Veriyi birbiriyle kıyaslamadan sıralamak asimptotik sınırı aşmamızı sağlar.
- Bu algoritmalar verinin sayısal özelliklerini ve dağılımını kullanır.
- Doğrusal zaman karmaşıklığı $O(n)$ seviyesine ulaşmak mümkündür.
- Belirli kısıtlamalar (sayı aralığı, bellek) altında çok hızlıdır.

Counting Sort: Sayma Mantığı

- Elemanların frekanslarını sayarak doğrudan pozisyonlarını belirleyen bir yöntemdir.
- Girdi değerleri k gibi sınırlı bir tamsayı aralığında olmalıdır.
- Sayıların adetlerini tutmak için $O(k)$ boyutunda yardımcı dizi kullanılır.
- Karşılaştırma yapmadığı için $O(n + k)$ sürede çalışır.

Counting Sort

Elemanları değerlerine göre sayar ve kümülatif toplam kullanarak her elemanın sıralı dizideki kesin yerini belirler.

Time Complexity: $O(n + k)$

Input Array

8	3	0	7	3	9	1	0	9	9	4	9
---	---	---	---	---	---	---	---	---	---	---	---

Count Array (k=10)

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Output Array

-	-	-	-	-	-	-	-	-	-	-	-
---	---	---	---	---	---	---	---	---	---	---	---

Play

Ready

Counting Sort: Yardımcı Dizi ve Kümülatif Toplam

- Count dizisinin kümülatif toplamı, her sayının bitiş indisini verir.
- Diziyi sondan başa taramak algoritmayı 'Kararlı' (Stable) kılar.
- Kararlılık, aynı değerdeki elemanların başlangıç sırasını korumasını sağlar.
- Karmaşık verileri (nesneleri) sıralamak için bu özellik hayattır.

Counting Sort: Zaman ve Bellek Analizi

- Toplam zaman karmaşıklığı $O(n + k)$ asimptotik sınıfındadır.
- Eğer $k = O(n)$ ise, algoritma tam lineer sürede biter.
- Bellek kullanımı giriş boyutu ve değer aralığına $(n + k)$ bağlıdır.
- k çok büyükse (örneğin $k = n^2$), verimlilik karesel algoritmaya düşer.

Radix Sort: Basamak Basamak Sıralama

- Sayıları basamak basamak (hanelerine göre) sıralayan bir stratejidir.
- Her bir basamak için 'Kararlı' bir alt algoritma kullanılır.
- Genellikle alt algoritma olarak Counting Sort tercih edilir.
- Çok büyük sayıların doğrusal sürede sıralanmasını sağlar.

Diagram

Radix Sort

Sayıları basamaklarına (birler, onlar, yüzler...) göre sırayla gruplandırarak sıralar. Her basamakta "Counting Sort" gibi kararlı bir yöntem kullanır.

Time Complexity: $O(d * (n + k))$

Current Digit: Units (1s)

782 812 827

660 569

449 411

271 298

033

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Play

```
radix_sort.py
def radix_sort(arr):
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        counting_sort_by_digit(arr, exp)
        exp *= 10
```

LSD vs MSD Radix Sort

- LSD: En düşük basamaktan başlar, genel sayı sıralamada yaygındır.
- MSD: En yüksek basamaktan başlar, sözlük sırası (string) için idealdir.
- MSD özyinelemeli (recursive) bölme mantığıyla çalışabilir.
- Veri türüne göre basamak tarama yönü performansı belirler.

Radix Sort: Karmaşıklık Analizi

- Karmaşıklık $O(d \cdot (n + k))$ olarak hesaplanır (d : basamak sayısı).
- d sabitse algoritma $O(n)$ doğrusal zaman performansına ulaşır.
- Karşılaştırma tabanlı $O(n \log n)$ bariyerini pratik uygulamalarda aşar.
- Sabit uzunluklu anahtarlar (IP, Plaka) için en verimli yoldur.

Bucket Sort: Kova Dağılımı

- Veriyi belirli aralıklara (kovalara) dağıtarak parçalı sıralama yapar.
- Verinin 0, 1 aralığında üniform dağıldığı varsayılır.
- Her kova kendi içinde basit bir algoritmayla (Insertion) sıralanır.
- Dağıt ve Birleştir (Scatter-Gather) felsefesini kullanır.

Diagram

Bucket Sort

Elemanları ırlı aralıkları temsil eden kovalara dağıtır (Scatter). Her bir kovayı kendi içinde sıralar ve ardından hepsini birleştirir (Gather).

Time Complexity: $O(n + k)$

Stage: Scattering...

0.63

0.98

0.16

0.09

0.94

0.32

0.25

0.80

0.54

0.67

[0.0 - 0.2]

[0.2 - 0.4]

[0.4 - 0.6]

[0.6 - 0.8]

[0.8 - 1.0]

Play

```
bucket_sort.py
def bucket_sort(arr):
    buckets = [[] for _ in range(k)]
    for x in arr: # Scatter
        buckets[idx(x)].append(x)
    for b in buckets: b.sort() # Sort
    return concatenate(buckets) # Gather
```

Bucket Sort: Ortalama ve En Kötü Durum

- Veri kovalara eşit dağıldığında ortalama süre $O(n)$ 'dir.
- Tüm veriler tek bir kovaya dolarsa süre $O(n^2)$ 'ye düşer.
- Veri dağılımının istatistiksel düzgünlüğü başarının temel anahtarıdır.
- Olasılıksal analizde en verimli doğrusal yaklaşımlardan biridir.

Bucket Sort Uygulama Alanları

- Ondalıklı sayıların ve yoğunluk haritalarının hızlı düzenlenmesinde kullanılır.
- Grafik motorlarında derinlik (z-buffer) sıralaması için idealdir.
- Dağıtık sistemlerde yük dengeleme (load balancing) için temeldir.
- Veri bilimi projelerinde ön-işleme (preprocessing) aşamasında tercih edilir.

Comparison vs Non-Comparison Summary

- Karşılaştırmaz yöntemler $O(n)$ ile hız rekoru kırarlar.
- Ancak bellek kullanımı ve veri kısıtlamaları (k, d) dezavantajdır.
- Karşılaştırmalı yöntemler (Quick/Merge) her türlü veriye uyumludur.
- Mühendislik seçimi 'Hız vs Esneklik' dengesine dayanır.

Hibrit Yaklaşımlara Giriş

- Tek bir algoritma her zaman en iyi sonucu vermez.
- Hibrit sistemler veri boyutu ve türüne göre strateji değiştirir.
- Yazılım kütüphaneleri performans garantisi için hibritleri standartlaştırmıştır.
- Amaç en kötü durum riskini yönetip ortalama hızı artırmaktır.

Timsort: Modern Dünyanın Sıralama Algoritması

- Python ve Java'nın varsayılan sıralama motorudur.
- Merge Sort'un logaritmik gücüyle Insertion Sort'un hızını birleştirir.
- Verideki doğal sıralı blokları (runs) tespit ederek çalışır.
- Adaptif, kararlı (stable) ve yüksek performanslı bir canavardır.

Diagram

Introsort: C++ Standart Kütüphanesi

- C++ STL içindeki `std::sort()` fonksiyonunun temelidir.
- Quick Sort ile başlar, derinlik artınca Heap Sort'a geçer.
- Küçük veri parçalarında ($n < 16$) Insertion Sort'u tetikler.
- Garantili $n \log n$ hızıyla Quick Sort'un risklerini yok eder.

External Sorting: Belleğe Sığmayan Veriler

- RAM'e sığmayan terabaytlarca veriyi diskte sıralama tekniğidir.
- Veri parçalara bölünür, tek tek sıralanır ve birleştirilir.
- Disk I/O maliyeti işlemci süresinden daha kritik bir faktördür.
- Veritabanı motorlarının (DBMS) temel operasyonel gücüdür.

Counting Sort Pseudocode İncelemesi

- Üç dizi yönetimi: Girdi (A), Çıktı (B) ve Sayaç ©.
- Kümülatif toplam adımı elemanın indisini matematiksel olarak hesaplar.
- For döngüsü sondan başlar; bu detay 'kararlılık' için kritiktir.
- Kodun her satırı doğrusal zaman maliyetine katkı sağlar.

Algorithm

```
1  procedure counting_sort(A, B, k):
2      let C[0..k] be a new array
3      for i = 0 to k:
4          C[i] = 0
5      for j = 1 to length(A):
6          C[A[j]] = C[A[j]] + 1
7      // C[i] now contains the number of elements equal to i
8      for i = 1 to k:
9          C[i] = C[i] + C[i-1]
10     // C[i] now contains the number of elements less than or equal to i
11     for j = length(A) downto 1:
12         B[C[A[j]]] = A[j]
13         C[A[j]] = C[A[j]] - 1
```

<https://gemini.google.com/share/81fa62c2794c>

Radix Sort Pseudocode İncelemesi

- Hanelerin sayısı (d) kadar dönen basit bir dögüdür.
- İçerideki 'Stable Sort' çağrısı algoritmanın bütünlüğünü sağlar.
- Mod ve Bölme işlemleriyle istenen basamak değeri çekilir.
- Karmaşık anahtarları basitleştirerek çözen modüler bir yapıdır.

Algorithm

```
1  procedure radix_sort(A, d):  
2      for i = 1 to d:  
3          // use a stable sort (like Counting Sort)  
4          // to sort array A on digit i  
5          counting_sort_on_digit(A, i)
```



<https://gemini.google.com/share/c38520657d22>

Radix sort anlatiklari

Bucket Sort Pseudocode İncelemesi

- Bir dizi liste (linked lists) oluşturularak kovalar hazırlanır.
- Elemanlar matematiksel bir fonksiyonla ilgili kovaya fırlatılır.
- Her liste kendi içinde ayrı ayrı (sıralanmış halde) yönetilir.
- Son aşamada listeler peş peşe eklenerek sonuç üretilir.

Algorithm

```
1  procedure bucket_sort(A):
2      n = length(A)
3      let B[0..n-1] be a new array of empty lists
4      for i = 1 to n:
5          insert A[i] into list B[floor(n * A[i])]
6      for i = 0 to n-1:
7          sort list B[i] with insertion sort
8      concatenate lists B[0], B[1], ..., B[n-1] together
```

<https://gemini.google.com/share/20aa4b9aae4a>

Sıralama Algoritmaları Karar Ağacı

- Sayısal aralık küçükse ($k < n$) Counting Sort seçilir.
- Kararlılık gerekmiyorsa ve RAM azsa Quick Sort idealdir.
- Veri diskten okunuyorsa Merge Sort mimarisi zorunludur.
- Gerçek üretim ortamlarında her zaman hibritler (Timsort) kullanılır.

Veri Dağılımının Algoritma Seçimine Etkisi

- Üniform dağılım Bucket Sort'u dünyanın en hızlısı yapar.
- Rastgele veri Quick Sort için mükemmel bir oyun alanıdır.
- Sıralı veri Insertion Sort'u Ferrari hızına taşır.
- Çarpık (skewed) dağılımlarda Timsort en güvenli seçimdir.

Bellek Hiyerarşisi (Cache) Etkileri-2

- $O(n)$ olan Counting Sort, rastgele erişimle yavaşlayabilir.
- $O(n \log n)$ olan Quick Sort, Cache Hit ile hızlanabilir.
- Asimptotik hıza rağmen donanım gecikmeleri performansı belirler.
- Mühendisler cache dostu indisleme yapılarına odaklanmalıdır.

Algoritma Görselleştirme: Radix ve Bucket

- Radix: Sayıların basamak basamak 'dans ederek' dizilmesini izlemek.
- Bucket: Verinin kutulara dolup sonra düzenli akmasını görmek.
- Görsel modeller, karmaşık kod mantığının zihinde kalıcı olmasını sağlar.
- Mekanizmanın ruhunu anlamak kodu ezberlemekten değerlidir.

Gelecekteki Sıralama: Quantum ve Paralel

- Çok çekirdekli sistemlerde paralel Merge/Quick Sort devrimi.
- GPU (CUDA) mimarilerinde binlerce verinin anlık sıralanması.
- Kuantum bilgisayarlarda arama ve sıralama limitlerinin değişimi.
- Teknolojinin sınırları algoritmik düşüncüyü sürekli evrimleştirir.

Hafta 4 Özeti ve 1. Aşama Kapanışı

- Sıralama dünyasındaki tüm asimptotik sınıfları öğrendik.
- Teorik bilgiyi pratik kütüphane tasarımlarıyla birleştirdik.
- Haftaya: Arama algoritmaları ve Graf dünyasına giriş yapıyoruz.
- Algoritma analizinde 1. aşamayı başarıyla tamamladık.

Question (Medium)

Counting Sort algoritmasının 'kararlı' (stable) kalması için neden diziyi sondan başa doğru taramamız gerekir?

Solution Steps

- Count dizisi her eleman için son pozisyonu saklar
- Sondan başa tarandığında, aynı değere sahip elemanlardan sonuncusu hedef dizideki son pozisyonuna yerleşir
- Bu sayede orijinal sıradaki elemanların göreceli konumları korunur
- Baştan sona tarama yapılırsa ilk eleman son pozisyona gider ve kararlılık bozulur

Learning Outcome: Algoritma kararlılığı ve indis yönetimi

Soru 1 (Counting Sort ve Kararlılık)

Counting Sort algoritmasında, sıralanmış diziyi oluştururken neden girdi dizisini sondan başa doğru taramamız gerekir?

Çözüm Adımları

- **Birikimli Toplam (Cumulative Count):** Yardımcı `count` dizisi, her elemanın hedef dizide yerleşeceği **en son (sağdaki)** pozisyonun indisini saklar.
- **Sondan Başa Tarama:** Girdi dizisini sondan başa taradığımızda, aynı değere sahip elemanlardan en sağda olanı, `count` dizisindeki son müsait pozisyona yerleşir.

- **Konumun Korunması:** Bu sayede orijinal dizide daha sağda olan eleman, sıralanmış dizide de daha sağda kalır. Bu da **Kararlılık (Stability)** özelliğini sağlar.
- **Hatalı Senaryo (Baştan Sona):** Eğer baştan başlasaydık, orijinal dizinin en solundaki eleman, `count` dizisindeki son (en sağdaki) pozisyona gidecek ve göreceli sıra tersine dönecekti.
- **Sonuç:** Counting Sort'un kararlı kalması, Radix Sort gibi bu algoritmayı temel alan diğer sıralama yöntemlerinin doğru çalışması için hayati önem taşır.

Learning Outcome: Algoritma kararlılığı ve indis yönetimi

Soru 2 (Parametrik Karmaşıklık Analizi)

Bir veri setinde $n=100$ eleman var ve değerler $[0, 10000]$ aralığında ise **Counting Sort** mu yoksa **Quick Sort** mu kullanmak daha mantıklıdır?

Analiz Adımları

- **Parametreler:** Eleman sayısı $n = 100$, değer aralığı (skala) $k = 10000$ olarak verilmiştir.
- **Counting Sort Analizi:** Karmaşıklığı $O(n + k)$ 'dir. Bu senaryoda yaklaşık $100 + 10000 = 10100$ birimlik bir işlem ve devasa bir yardımcı dizi (bellek) maliyeti oluşur.
- **Quick Sort Analizi:** Ortalama karmaşıklığı $O(n \log n)$ 'dir. $100 \cdot \log_2(100) \approx 100 \cdot 6.64 \approx 664$ birimlik bir işlem yükü oluşur.
- **Karşılaştırma:** $O(n \log n)$, bu özel durumda $O(n + k)$ 'dan yaklaşık **15 kat** daha hızlı sonuç verir.
- **Sonuç:** k değeri n 'den çok büyük ($k \gg n$) olduğu için **Quick Sort** hem zaman hem de bellek açısından çok daha verimlidir.

Learning Outcome: Veri aralığına (k) göre algoritma seçimi

Question (Hard)

Bucket Sort'un en kötü durum (worst-case) analizi nedir ve bu durum nasıl oluşur?

Solution Steps

- Tüm verinin tek bir kovaya dolması (clustering) durumudur
- Bu durumda performans, kovayı sıralayan algoritmaya (genelde insertion sort) bağlı kalır
- Sonuç: $O(n^2)$ karesel karmaşıklık
- Çözüm için iyi bir hash fonksiyonu veya üniform dağılım gereklidir

Learning Outcome: Dağılım tabanlı algoritma analizi

Soru 3 (İleri Seviye Dağılım Analizi)

Bucket Sort algoritmasının "en kötü durum" (worst-case) analizi nedir ve bu durum nasıl oluşur?

Çözüm Adımları

- **Kümelenme (Clustering):** En kötü durum, seçilen kova (bucket) sayısı ne olursa olsun, tüm verilerin tek bir kovaya dolmasıyla oluşur.
- **Algoritma Bağımlılığı:** Veriler tek bir kovada toplandığında, performans o kovayı sıralamak için kullanılan iç algoritmaya (genellikle **Insertion Sort**) bağlı kalır.
- **Zaman Karmaşıklığı:** Insertion Sort, n elemanlı tek bir kova için en kötü durumda $O(n^2)$ (Karesel Zaman) ile çalışır.
- **Neden Oluşur?** Kötü bir hash fonksiyonu seçimi veya girdi verilerinin üniform (düzenli) dağılmaması bu duruma yol açar.
- **Çözüm:** Verinin karakteristiğine uygun bir dağılım fonksiyonu seçilmeli veya kova içi sıralama için $O(n \log n)$ garantili algoritmalar tercih edilmelidir.